

P3973R0

bit_cast_as: Element type reinterpretation for std::simd

Draft Proposal, 2026-01-19

Issue Tracking:

[Inline In Spec](#)

Author:

[Daniel Towner](#) (Intel Corporation)

Audience:

LEWG, SG1

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

We propose `std::bit_cast_as()`, a facility for reinterpreting `std::simd` objects at different element granularities. This enables safe, efficient operations like converting packed bytes to shorts, or float vectors to their underlying bit patterns, with compile-time size verification and automatic element count inference. The design naturally generalizes to other contiguous containers, which we present as design options for committee feedback. This paper was originally part of [P3445R0] which applied to `simd` only, but is now split out as a focused and generalised proposal. This paper relies on [P3983R0] for the array-like layout guarantees that make the implementation portable and safe.

Table of Contents

- 1 **Introduction**
- 2 **Revision History**
- 3 **Motivation**

- 3.1 The SIMD Element Reinterpretation Problem
 - 3.1.1 Signal Processing - Packed Sample Conversion
 - 3.1.2 Bit Manipulation - Type Punning
 - 3.1.3 Data Packing - Bytes to Words
- 3.2 Intrinsics Already Support This Pattern
- 3.3 Why `std::bit_cast` Doesn't Solve This
- 3.4 What We Want

4 Proposed Solution

- 4.1 Usage Examples
- 4.2 Relationship to Existing Facilities

5 Design Decisions

- 5.1 Element Count Inference
- 5.2 Size Mismatch Handling
- 5.3 Abi Computation Strategy
- 5.4 Implementation Mechanism
- 5.5 Naming

6 Implementation Experience

- 6.1 Conceptual Implementation

7 Generalization to Other Types

- 7.1 Natural Extension to `std::array`
- 7.2 Natural Extension to `std::span`
- 7.3 Unified Design Pattern

8 Design Alternatives Considered

- 8.1 Alternative: Make this a member function
- 8.2 Alternative: Use explicit count parameter

9 Wording

- 9.1 Header `<simd>` synopsis additions
- 9.2 `bit_cast_as` for `simd` [`simd.bit_cast_as`]
- 9.3 Feature Test Macro

Issues Index

§ 1. Introduction

SIMD programming frequently requires reinterpreting vector data at different element granularities—converting packed bytes to shorts, accessing the bit representation of floats, or regrouping data for different operations. While platform intrinsics have long supported this pattern naturally, with `std::simd` programmers must use `std::bit_cast` with fully-specified target types, manually computing element counts and constructing appropriate ABIs.

This proposal introduces `std::bit_cast_as<T>()`, a facility that brings `std::simd` to parity with platform intrinsics by automatically inferring element counts when reinterpreting SIMD vectors.

Instead of writing:

```
// Verbose: must specify element count and worry about ABI selection
auto shorts = std::bit_cast<vec<uint16_t, 8>>(bytes);
```

Programmers can write:

```
// Clear: element count inferred automatically
auto shorts = std::bit_cast_as<uint16_t>(bytes);
```

The facility provides compile-time safety through automatic size verification, eliminates error-prone manual count calculations, and makes the intent explicit. The design naturally generalizes to other contiguous containers like `std::array` and `std::span`, which we present as design options for LEWG feedback.

This proposal requires array-like layout guarantees being developed for `std::simd`, and those have wider implications which are discussed in [P3983R0]. It represents a focused improvement to make element reinterpretation as natural and safe in portable C++ as it already is in platform-specific intrinsics.

§ 2. Revision History

R0 - Initial revision

§ 3. Motivation

§ 3.1. The SIMD Element Reinterpretation Problem

SIMD programming frequently requires reinterpreting vector data at different element granularities. Common scenarios include:

§ 3.1.1. Signal Processing - Packed Sample Conversion

```
// Receive 16 packed 8-bit samples
vec<uint8_t, 16> samples_8bit = receive_audio();

// Need to process as 8 16-bit samples
// Current approach: verbose and error-prone
auto samples_16bit = std::bit_cast<vec<uint16_t, 8>>(samples_8bit);
// Must manually specify element count      ^^^
```

§ 3.1.2. Bit Manipulation - Type Punning

```
// Get bit representation of floats for IEEE 754 operations
vec<float, 8> floats = /*...*/;

// Current: must know exact result type
auto bits = std::bit_cast<vec<uint32_t, 8>>(floats);
//      ^^^ magic number
```

§ 3.1.3. Data Packing - Bytes to Words

```
// Combine byte pairs into 16-bit values
vec<uint8_t, 32> bytes = load_packed_data();

// Current: compute count manually, specify Abi explicitly
using target_abi = /* ??? */;
auto words = std::bit_cast<vec<uint16_t, 16>>(bytes);
```

§ 3.2. Ininsics Already Support This Pattern

Platform intrinsics have long supported element reinterpretation without needing to specify counts:

```
// x86 intrinsics - type changes, same register
__m256i bytes_vec = _mm256_loadu_si256(/*...*/);
__m256i shorts_vec = bytes_vec; // Same bits, different interpretation

// Explicit reinterpret intrinsics
__m256 floats = _mm256_loadu_ps(/*...*/);
__m256i bits = _mm256_castps_si256(floats); // float[8] → int32[8]
```

The platform already does this naturally but `std::simd` makes it awkward.

§ 3.3. Why `std::bit_cast` Doesn't Solve This

While `std::bit_cast` handles type reinterpretation, it requires explicitly specifying the target type including element count:

```
// bit_cast requires spelling out the complete type
auto shorts = std::bit_cast<std::simd<uint16_t, 8>>(bytes);
//                                     ^^^ must specify
```

This is particularly problematic for generic code where the element count must be computed:

```
template<typename NewT, typename T, typename Abi>
auto reinterpret_elements(basic_vec<T, Abi> v) {
    constexpr size_t old_count = basic_vec<T, Abi>::size();
    constexpr size_t new_count = old_count * sizeof(T) / sizeof(NewT);
    using new_vec = resize_t<new_count, rebind_t<NewT, basic_vec<T, Abi>>>
    return std::bit_cast<new_vec>(v);
}
```

Problems with this approach:

- **Verbose:** Must manually compute element count and construct Abi
- **Error-prone:** Possible to get count calculation wrong
- **Unclear:** The operation is "reinterpret elements" but code doesn't say that

- **Fragile:** Changes to one type parameter require recalculating everything

A natural solution is to provide a standard facility that automates this pattern, eliminating the manual computation and potential for error.

§ 3.4. What We Want

```
vec<uint8_t, 16> bytes = receive_data();

// Clear, safe, concise
auto shorts = std::bit_cast_as<uint16_t>(bytes);
// Returns vec<uint16_t, 8> - count inferred automatically
```

Benefits:

- **Automatic:** Element count computed from sizes
- **Safe:** Compile-time verification that sizes match
- **Clear:** Intent is obvious
- **Generic:** Works in templates without manual ABI computation

§ 4. Proposed Solution

We propose adding `std::bit_cast_as<T>()` to `<simd>`:

```
namespace std {
    template<typename T, typename U, typename Abi>
    basic_vec<T, /* computed Abi */> bit_cast_as(const basic_vec<U, Abi>& v)
}
```

Effect: Returns a `simd` object with element type `T` containing the same bits as `v`, with element count automatically inferred.

Constraints:

- Sizes must match exactly: `sizeof(U) * simd<U, Abi>::size() == sizeof(T) * new_count`
- Result type must be valid: `basic_vec<T, computed_Abi>` must be well-formed

§ 4.1. Usage Examples

```
// basic element reinterpretation
vec<uint8_t, 16> bytes = /*...*/;
auto shorts = std::bit_cast_as<uint16_t>(bytes);    // vec<uint16_t, 8>
auto ints    = std::bit_cast_as<uint32_t>(bytes);    // vec<uint32_t, 4>
auto longs   = std::bit_cast_as<uint64_t>(bytes);    // vec<uint64_t, 2>
```

```
// Float/int type punning
vec<float, 8> floats = /*...*/;
auto bits = std::bit_cast_as<uint32_t>(floats);    // Access bit representat

// Manipulate bits
bits &= 0x7FFFFFFF;    // Clear sign bit

// Convert back
auto abs_floats = std::bit_cast_as<float>(bits);
```

```
// Generic SIMD code
template<typename T, typename Abi>
auto as_bytes(const basic_vec<T, Abi>& v) {
    return std::bit_cast_as<std::byte>(v);
}

template<typename T, typename U, typename Abi>
auto convert_elements(const basic_vec<U, Abi>& v) {
    return std::bit_cast_as<T>(v);
}
```

```
// Compile time safety
vec<uint8_t, 15> odd_size = /*...*/;

// Error: 15 bytes doesn't evenly divide into uint32_t
auto bad = std::bit_cast_as<uint32_t>(odd_size);    // Won't compile
```

§ 4.2. Relationship to Existing Facilities

Compared to `std::bit_cast`:

- `bit_cast` requires fully specifying target type including count
- `bit_cast_as` infers count automatically
- `bit_cast_as` uses existing simd type machinery (`rebind_t`, `resize_t`) in valid ways
- More ergonomic for the common "reinterpret elements" use case

Compared to intrinsics:

- Intrinsics already support this naturally: `_mm256_castps_si256(vec)`
- `std::simd` should provide equivalent expressiveness
- But with better type safety and generic programming support

§ 5. Design Decisions

§ 5.1. Element Count Inference

We propose that element count should be automatically inferred from the sizes.

```
// User specifies only element type
auto result = bit_cast_as<uint16_t>(vec);

// NOT: bit_cast_as<uint16_t, 8>(vec) // Explicit count rejected
```

The rationale for automatic inference:

- Safer - prevents count/size mismatches
- More concise - especially in generic code
- Matches how intrinsics work (`_mm256_castps_si256` doesn't take a count)
- Matches `std::bit_cast` philosophy (sizes determine validity)
- No use case where explicit count adds value

§ 5.2. Size Mismatch Handling

We require exact size match with a compilation error otherwise:

```
vec<uint8_t, 15> vec;  
auto bad = bit_cast_as<uint32_t>(vec); // Error: 15 bytes != N * 4 bytes
```

The rationale for this is:

- Matches `std::bit_cast` behavior (requires `sizeof(From) == sizeof(To)`)
- Prevents silent data loss (no truncation)
- Prevents undefined behavior (no padding/uninitialized data)
- Explicit operations available if truncation is desired
- No need for runtime support - this is fundamentally a compile-time operation

§ 5.3. Abi Computation Strategy

We can specify the required ABI changes which arise from changing the element type in terms of existing features from the draft standard, namely `rebind_t` and `resize_t`:

```
template<typename T, typename U, typename Abi>  
auto bit_cast_as(const basic_vec<U, Abi>& v) {  
    constexpr size_t old_count = simd<U, Abi>::size();  
    constexpr size_t new_count = old_count * sizeof(U) / sizeof(T);  
  
    // Step 1: Rebind to new element type  
    using new_type = rebind_t<T, basic_vec<U, Abi>>;  
  
    // Step 2: Resize to new element count  
    using new_vec = resize_t<new_count, new_type>;  
  
    return new_vec{/* bit reinterpretation */};  
}
```

Rationale:

- Reuses existing type machinery from draft standard.

- Yields a well-formed result when `rebind_t`/`resize_t` are well-formed, with ABI/layout correctness discussed in the issue below.

ISSUE 1 Is `resize(rebind(...))` always correct, or should it be `rebind(resize(...))`? Are they always equivalent?

§ 5.4. Implementation Mechanism

With the array-like layout guarantee of [P3983R0], the implementation could be straight-forward:

```
template<typename T, typename U, typename Abi>
auto bit_cast_as(const basic_vec<U, Abi>& v) noexcept {
    constexpr size_t new_count = basic_vec<U, Abi>::size() * sizeof(U) / s:
    using new_vec = resize_t<new_count, rebind_t<T, Abi>>;
    return std::bit_cast<new_vec>(v);
}
```

However, without guaranteed array-like layout (contiguous elements, no padding), `bit_cast` between simd types is not portable. Different ABIs could:

- Store elements in different orders
- Insert padding between elements
- Use platform-specific representations

[P3983R0] would guarantee that simd elements are stored contiguously in element order, making `bit_cast` safe and portable.

Platform intrinsics already support this because they make implicit array-like guarantees. This proposal brings `std::simd` to parity with intrinsics.

§ 5.5. Naming

`bit_cast_as` was chosen because:

- **Clear relationship to existing bit cast:** Immediately signals this is a bit-level reinterpretation, not a conversion

- **Distinguishes from conversions:** Unlike `as_bytes` (which is a view operation for `span`), this is specifically about bit-casting with automatic type inference
- **Explicit about the operation:** Makes it clear we're doing something at the bit level, not just changing element granularity semantically
- **Natural extension of existing vocabulary:** The `_as<T>` suffix pattern indicates "interpret as T" with automatic type deduction
- **Generalizes well:** If extended to `array`, `bit_cast_as<T>(arr)` clearly means "bit_cast this array, interpreting as elements of type T"

Another strong contender for the name was `as_elements`:

- While shorter and matching the `as_bytes` pattern from `span`, it doesn't clearly convey that this is a bit-level reinterpretation
- `as_elements` could be confused with accessing or viewing elements, rather than reinterpreting bits
- The relationship to `bit_cast` is less obvious, making it harder for users to understand the operation's guarantees and constraints

A few weaker alternatives were also considered:

- `simd_bit_cast<T>` - Our original name from the previous paper [P3445R0], but it is too narrow and doesn't indicate element reinterpretation. `std::simd` also dropped the `simd_` prefix in favour of a namespace, and applying that change here results in plain `bit_cast` as the name, which is clearly a bad name.
- `reshape_as<T>` - unclear whether it changes dimensions or element type
- `reinterpret_as<T>` - matches `reinterpret_cast` but less specific

§ 6. Implementation Experience

In Intel's implementation of `std::simd` the original element bit casting function called `simd_bit_cast` was added very early on because it is so widely used.

The implementation of Intel's `std::simd` library itself uses the element bit casting to make it easier to interface to compiler intrinsics. Intrinsics often require particular data types to be used to achieve certain effects, and the bit-cast allows the underlying bits to be quickly and easily reinterpreted.

Intel uses `std::simd` in a number of internal software projects, and some of those (particularly wireless or packet-processing) need to be able to easily reinterpret the underlying bits in different ways.

Some of those software projects were originally written in plain intrinsics and then rewritten to use `std::simd`. In those projects intrinsics like `_mm256_castps_si256` were used, and `bit_cast_as` provides the natural equivalent.

§ 6.1. Conceptual Implementation

```
template<typename T, typename U, typename Abi>
auto bit_cast_as(const simd<U, Abi>& v) noexcept {
    constexpr size_t old_bytes = sizeof(U) * simd<U, Abi>::size();
    constexpr size_t new_count = old_bytes / sizeof(T);
    static_assert(old_bytes % sizeof(T) == 0, "Size mismatch");

    using new_abi = simd_abi::resize_t<new_count, simd_abi::rebind_t<T, Abi>>();

    return std::bit_cast<simd<T, new_abi>>(v);
}
```

§ 7. Generalization to Other Types

§ 7.1. Natural Extension to `std::array`

The same operation makes sense for `std::array`:

```
std::array<uint8_t, 16> bytes = /*...*/;
auto shorts = std::bit_cast_as<uint16_t>(bytes); // Returns array<uint16_t, 8>
```

Implementation would be:

```
template<typename T, typename U, size_t N>
requires (sizeof(U) * N % sizeof(T) == 0)
constexpr array<T, sizeof(U) * N / sizeof(T)>
bit_cast_as(const array<U, N>& a) noexcept {
    return std::bit_cast<array<T, sizeof(U) * N / sizeof(T)>>(a);
}
```

Use cases:

- Protocol parsing (reinterpret byte arrays as structured data)
- File format handling
- Generic bit manipulation
- Compile-time data transformation

§ 7.2. Natural Extension to `std::span`

For `std::span`, the operation returns a view (not a copy):

```
std::span<int, 8> ints = /*...*/;
auto bytes = std::bit_cast_as<std::byte>(ints); // Returns span<byte, 32>
```

Implementation would be:

```
template<typename T, typename U, size_t Extent>
    requires (Extent != dynamic_extent) &&
             (sizeof(U) * Extent % sizeof(T) == 0)
constexpr span<T, sizeof(U) * Extent / sizeof(T)>
    bit_cast_as(span<U, Extent> s) noexcept {
    return {reinterpret_cast<T*>(s.data()), sizeof(U) * Extent / sizeof(T)};
}
```

Key difference: Returns a *view* (span) rather than a *value* (copy), consistent with span semantics.

****Relationship to `std::as_bytes`****

- `std::as_bytes` is already in C++20: `span<byte> as_bytes(span<T> s)`
- `bit_cast_as` would generalize it: `as_bytes(s) ≡ bit_cast_as<std::byte>(s)`
- Could coexist, with `as_bytes` remaining for compatibility

§ 7.3. Unified Design Pattern

A unified design emerges:

- **Value types** (`simd`, `array`) → return values (copies with bitwise reinterpretation)
- **View types** (`span`) → return views (reinterpreted pointer + adjusted size)

- **Same operation**, different return category based on input

Consistency principle: Preserve the "value vs view" nature of the input.

§ 8. Design Alternatives Considered

§ 8.1. Alternative: Make this a member function

```
auto shorts = bytes.bit_cast_as<uint16_t>();
```

Rejected because:

- Inconsistent with `as_bytes` (free function)
- Harder to extend to multiple types later
- Free function pattern more flexible for ADL

§ 8.2. Alternative: Use explicit count parameter

```
auto result = bit_cast_as<uint16_t, 8>(bytes);
```

Rejected because:

- Redundant - count is deterministic from sizes
- Error-prone - user could specify wrong count
- Less convenient in generic code
- No added safety value
- Inconsistent with how intrinsics work

§ 9. Wording

Tentative wording for the initial proposal.

§ 9.1. Header `<simd>` synopsis additions

```
namespace std {  
    template<typename T, typename U, typename Abi>  
    simd<T, /* see below */> bit_cast_as(const simd<U, Abi>& v) noexcept;  
}
```

§ 9.2. `bit_cast_as` for `simd` [`simd.bit_cast_as`]

```
template<typename T, typename U, typename Abi>  
simd<T, /* see below */> bit_cast_as(const simd<U, Abi>& v) noexcept;
```

Constraints:

- `sizeof(U) * simd<U, Abi>::size() % sizeof(T) == 0`
- Let `new_count = sizeof(U) * simd<U, Abi>::size() / sizeof(T)`
- Let `NewAbi = simd_abi::resize_t<new_count, simd_abi::rebind_t<T, Abi>>`
- `simd<T, NewAbi>` is a valid, complete type

Mandates:

- `is_trivially_copyable_v<T>` is true
- `is_trivially_copyable_v<U>` is true

Returns: `bit_cast<simd<T, NewAbi>>(v)`

Remarks:

- This function shall not participate in overload resolution unless the constraints are satisfied.
- The bit representation interpretation requires that `simd` types have array-like layout as specified in P{array}.

§ 9.3. Feature Test Macro

Add to `<version>`:

```
#define __cpp_lib_simd_bit_cast_as 202601L // also in <simd>
```

§ Issues Index

ISSUE 1 Is `resize(rebind(...))` always correct, or should it be `rebind(resize(...))`? Are they always equivalent?

